COLLEGE:ABASAHEB GARWARE (AUTONOMOUS)

T.Y.BSc.(Computer Science)

PROJECT REPORT ON

Cybersecurity Shenstial Shield Project

ACADEMIC YEAR:2025-2026

SUBMITTED BY: SANJANA SUBHASH KAMBLE

1.INTRODUCTION

In the modern digital era, computer networks are vital for daily operations, but the rapid growth of internet usage has significantly increased risks like unauthorized access and malware. While organizations use firewalls and IDS to safeguard infrastructure, these tools are often expensive.

SentinelShield addresses this by providing a simple, Python-based system that runs on Windows Command Prompt (CMD). It is designed to demonstrate:

- **Network scanning** to detect active and unauthorized devices.
- **Intrusion detection** through monitoring system activities.
- **File integrity monitoring** to track changes in real-time.
- **Log generation** for security history and analysis

SentinelShield is a basic cybersecurity monitoring system developed in Python to bridge this gap. It serves as a simple, lightweight, and cost-effective solution designed specifically for the Windows environment. By running through the Command Prompt (CMD), it proves that effective network scanning, intrusion detection, and file integrity monitoring can be achieved without advanced penetration testing tools

2. System Requirements

To run the SentinelShield system effectively, the following hardware and software specifications are required:

Hardware Requirements:

- **Processor:** Intel i3 or higher.
- **RAM:** Minimum 4GB.
- **Hard Disk:** 20GB free space.

Software Requirements:

- **Operating System:** Windows 10/11.
- **Environment:** Python 3.x and Command Prompt (CMD).
- **Python Libraries:** scapy, watchdog, hashlib, logging, argparse, os, and time

3. Tools & Technology Used

Technology	Purpose
Python	Primary programming language used for logic and integration.
Scapy	Used for network scanning by broadcasting ARP packets.
Watchdog	Monitors file system events like creation, deletion, and modification.
Hashlib	Generates SHA-256 fingerprints to detect file tampering.
Logging	Records suspicious activities into .log files with timestamps.
CMD	Serves as the execution environment for the tool.

4. Advantages

Simplicity: The tool is easy to understand and specifically designed for students and beginners.

Platform Accessibility: It runs on standard Windows CMD, removing the need for Kali Linux.

Resource Efficient: It is a lightweight system with minimal hardware impact.

Real-Time Security: Provides immediate monitoring of local network devices and file changes.

5.Future Enhancements

To evolve SentinelShield into a more robust security suite, the following features are proposed:

- ✓ **User Interface:** Developing a GUI using Tkinter for better usability.
- ✓ **Alerting Systems:** Integrating Email and SMS notification systems for instant alerts.
- ✓ **Advanced Analytics:** Implementing a real-time dashboard and AI-based anomaly detection.
- ✓ **Data Management:** Cloud integration and a dedicated database logging system.
- ✓ **Active Defense:** Including an automatic blocking mechanism for malicious IPs.

6. Implementation Details (Code Overview)

```
# Conceptual Structure of sentinelshield.py


import scapy.all as scapy

from watchdog.observers import Observer

from watchdog.events import
FileSystemEventHandler

import hashlib

import logging


# 1. Network Scan Logic

def scan(ip_range):

    # Sends ARP request to the specified IP
range [cite: 174]

    # Returns list of discovered IP and MAC
addresses
```

```
    pass


# 2. File System Event Handling

class MyHandler(FileSystemEventHandler):

    def on_modified(self, event):

        # Triggered when file modification is
detected

        pass


    def on_deleted(self, event):

        # Triggered when a file is removed [cite:
176]

        pass


# 3. Integrity Check Logic

def calculate_hash(file_path):

    # Generates SHA-256 fingerprint for a file
[cite: 177]

    pass
```

4. Main Execution

if __name__ == "__main__":

    # Setup logging and start the Observer [cite: 170, 171]

    # Run scan() then enter a loop to monitor files

7.Module Breakdown & Logic

A. Network Scanning Module

This module utilizes the **Address Resolution Protocol (ARP)**. The system broadcasts ARP packets across a specified IP range; active devices respond with their MAC addresses, allowing the system to map the local network and flag unrecognized hardware.

B. File Monitoring & Hashing Module

The system uses the watchdog library to "watch" a directory.

> **on_modified():** Triggered when a file is changed.
>
> **calculate_hash():** When a change is detected, the module generates a new SHA-256 hash and compares it against the previous record to confirm if the file's integrity was compromised.

C. Logging Module

Every event (New device found, File modified, File deleted) is captured with a **Date and Time** stamp. This is stored in a file_audit.log, ensuring a persistent trail of evidence for security audits.

8.Infrastructure&Implementation

To build the **SentinelShield** system, you must follow a structured development lifecycle. Below is the step-wise infrastructure for executing the project:

Step 1: Environment Setup

Install Python: Ensure Python 3.x is installed on a Windows 10/11 environment.

Install Dependencies: Open CMD and install the necessary libraries using pip:

pip install scapy (for network scanning).

pip install watchdog (for file monitoring).

Directory Setup: Create a dedicated project folder and a specific sub-folder (e.g., C:/Monitor) to test file integrity.

Step 2: Developing the Network Scanning Module

Protocol Selection: Use the Address Resolution Protocol (ARP) to map the network.

Broadcasting: Write a function scan(ip_range) that sends ARP requests to a specific IP range (e.g., 192.168.1.0/24).

Data Extraction: Capture response packets to display a list of active IP and MAC addresses.

Step 3: Implementing File System Monitoring

Event Handling: Define a FileSystemEventHandler class to detect real-time changes.

Logic Implementation: Use on_modified() to catch edits and on_deleted() to catch removals.

Observer Start: Initialize the watchdog observer to continuously watch the target directory.

Step 4: Integrating Hashing for Integrity

Fingerprinting: Use hashlib to create a SHA-256 hash of critical files.

Comparison: Implement logic that compares the "old" hash with a "new" hash whenever a modification event is triggered to confirm tampering.

Step 5: Logging and Alerting

Audit Trail: Set up the logging module to save all events into a .log file.

Data Points: Ensure each log entry includes the date, time, event type (INFO/WARNING), and the file path involved.

Detailed Introduction

In the modern digital era, computer networks and information systems are the backbone of daily operations for organizations and individuals alike. However, the rapid growth of internet-connected devices has led to a surge in cyber threats, including unauthorized access, data breaches, and malicious file tampering. While professional

cybersecurity tools exist, many are expensive and require specialized Linux environments like Kali Linux.

SentinelShield is a basic cybersecurity monitoring system developed in Python to bridge this gap. It serves as a simple, lightweight, and cost-effective solution designed specifically for the Windows environment. By running through the Command Prompt (CMD), it proves that effective network scanning, intrusion detection, and file integrity monitoring can be achieved without advanced penetration testing tools.

Detailed Conclusion

SentinelShield serves as a successful proof-of-concept for a Windows-based security monitoring tool. Throughout its operation, the system effectively performs two critical functions: identifying active devices on a local network to prevent unauthorized access and monitoring file system events to ensure data integrity.

The project demonstrates that fundamental cybersecurity concepts—such as ARP scanning, SHA-256 hashing, and automated log analysis—can be implemented using accessible Python libraries. While it remains a beginner-level tool with limitations in blocking threats or scanning external networks, it provides a strong foundational framework for students and small organizations. Ultimately, SentinelShield highlights the importance of proactive monitoring in maintaining a secure digital infrastructure.

9.System Results and Output Analysis

The results of the **SentinelShield** project demonstrate its ability to successfully perform real-time monitoring and network discovery within a Windows environment. Below is a detailed breakdown of the system's execution results and the operational output (OP).

---

1. Network Scanning Results

When the user executes the scanning module, the system broadcasts ARP packets to identify active nodes.

> **Targeting:** The system prompts the user to enter a Target IP or Range (e.g., 192.168.1.0/24).
>
> **Discovery:** It successfully retrieves and displays the **IP Address** and **MAC Address** of every active device on the local network.
>
> **Unauthorized Detection:** By comparing results, the system can identify unknown or suspicious devices connected to the LAN.

---

2. File Monitoring & Integrity Results

The system monitors a specified directory for any file system events using the watchdog library.

> **Modification Alerts:** If a file’s content is changed, the system triggers an on_modified() event and alerts the user.

Integrity Verification: The hashing module calculates the SHA-256 fingerprint; if the new hash does not match the stored hash, it confirms file tampering.

3.Sample Log Output (file_audit.log)

The Logging Module maintains a security history by recording events with precise timestamps. A typical output in the file_audit.log file looks as follows:

Deletion/Creation: The system instantly detects when files are removed or new documents are added to the monitored path.

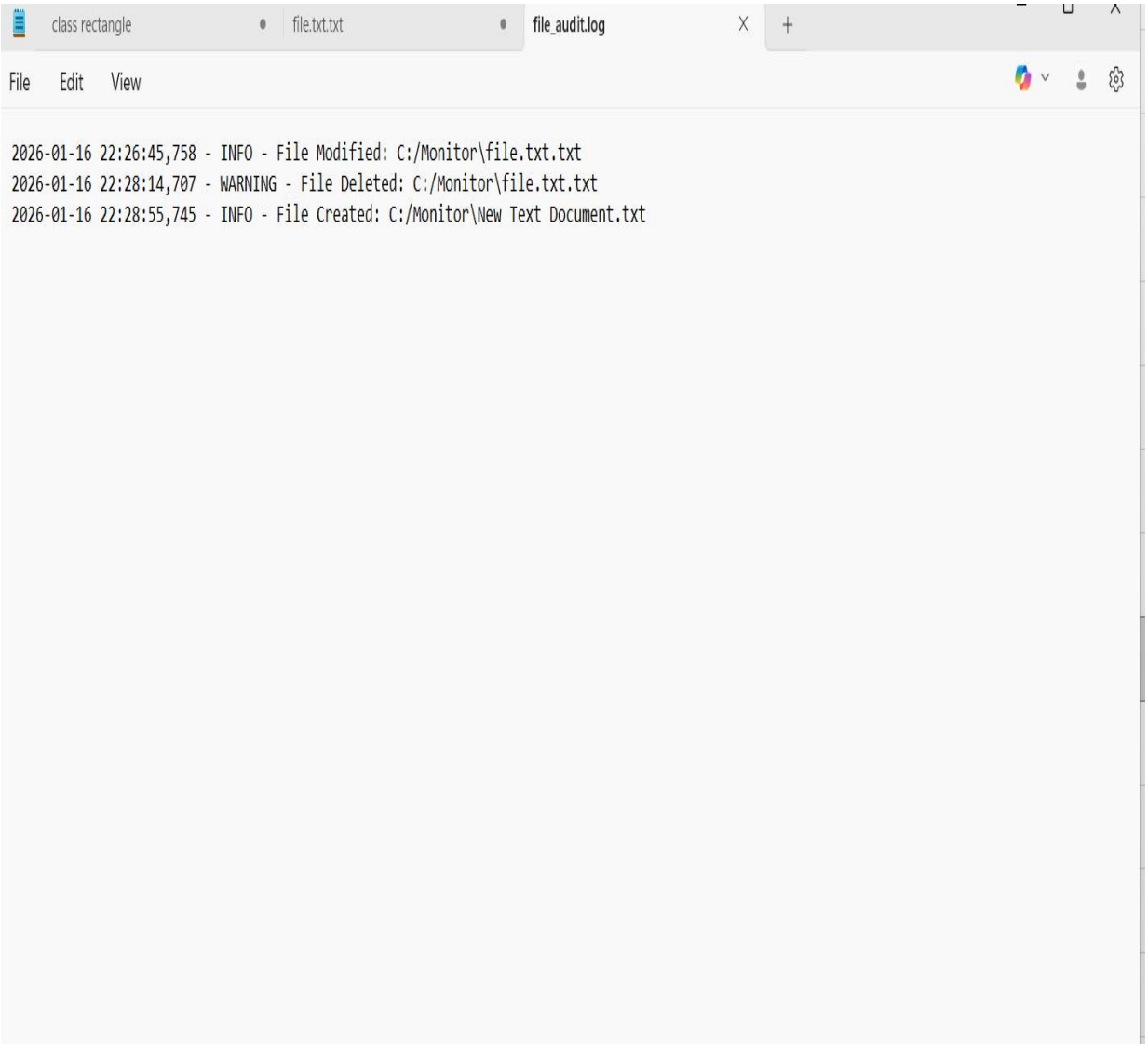
Timestamp	Severity	Event Description	File Path
2026-01-16 22:26:45	**INFO**	File Modified	C:/Monitor/file.txt.txt
2026-01-16 22:28:14	**WARNING**	File Deleted	C:/Monitor/file.txt.txt
2026-01-16 22:28:55	**INFO**	File Created	C:/Monitor/New Text Document.txt

4. Operational Performance (OP)

Environment: The system operates entirely within the **Windows Command Prompt (CMD)**, proving that Linux is not required for basic security tasks.

Resource Usage: It remains **lightweight**, running smoothly on systems with at least 4GB of RAM and an i3 processor

User Feedback: The system provides real-time command-line alerts, allowing for immediate awareness of network or file-level threats.



Administrator: Command Prompt

```
icrosoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.

:\Windows\System32> C:\Users\sanjana\OneDrive\Desktop\sops.py
nter Target IP or Range: 127.0.0.1
P Address              MAC Address
----------------------------------------

:\Windows\System32> C:\Users\sanjana\OneDrive\Desktop\sops.py
nter Target IP or Range: 127.0.0.1/32
P Address              MAC Address
----------------------------------------

:\Windows\System32> C:\Users\sanjana\OneDrive\Desktop\sops.py
nter Target IP or Range: 192.168.1.0/24
P Address              MAC Address
----------------------------------------
92.168.1.1             98:9d:b2:6e:00:53
92.168.1.36            d0:39:57:de:8b:b3

:\Windows\System32>
```

10. Conclusion

SentinelShield serves as a successful proof-of-concept for a Windows-based security monitoring tool. Throughout its operation, the system effectively performs two critical functions: identifying active devices on a local network to prevent unauthorized access and monitoring file system events to ensure data integrity.

The project demonstrates that fundamental cybersecurity concepts—such as ARP scanning, SHA-256 hashing, and automated log analysis—can be implemented using accessible Python libraries. While it remains a beginner-level tool with limitations in blocking threats or scanning external networks, it provides a strong foundational framework for students and small organizations. Ultimately, SentinelShield highlights the importance of proactive monitoring in maintaining a secure digital infrastructure.